	Result	Time	Cycles	GPU	SM Frequency	Process	Attributes
Current	7504 - rmsnorm_kernel (1, 1, 1)x(288, 1, 1)	3.17 usecond	5,040	0 - NVIDIA GeForce RTX 4060 Laptop GPU	1.59 cycle/nsecond	[11435] run	

GPU Speed Of Light Throughput

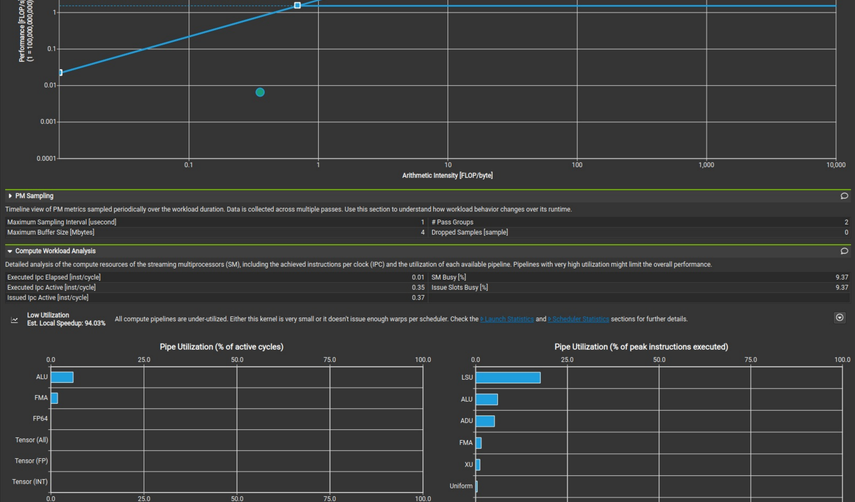
GPU Throughput Rooflines

High-level overview of the throughput for compute and memory resources of the GPU. For each unit, the throughput reports the achieved percentage of utilization with respect to the theoretical maximum. Breakdowns show the throughput for each individual sub-metric of Compute and Memory to clearly identify the highest contributor. High-level overview of the utilization for compute and memory resources of the GPU presented as a roofline chart.

Compute (SM) Throughput [%]	0.46	Duration [usecond]	3.17
Memory Throughput [%]	0.81	Elapsed Cycles [cycle]	5,040
L1/TEX Cache Throughput [%]	17.61	SM Active Cycles [cycle]	132
L2 Cache Throughput [%]	0.59	SM Frequency [cycle/nsecond]	1.59
DRAM Throughput [%]	0.81	DRAM Frequency [cycle/nsecond]	6.87

Small Grid This kernel grid is too small to fill the available resources on this device, resulting in only 0.0 full waves across all SMs. Look at Launch Statistics for more details.

Roofline Analysis The ratio of peak float (fp32) to double (fp64) performance on this device is 64:1. The kernel achieved close to 0% of this device's fp32 peak performance and 0% of its fp64 peak performance. See the Kernel Profiling Guide for more details on roofline analysis.



PM Sampling

Timeline view of PM metrics sampled periodically over the workload duration. Data is collected across multiple passes. Use this section to understand how workload behavior changes over its runtime.

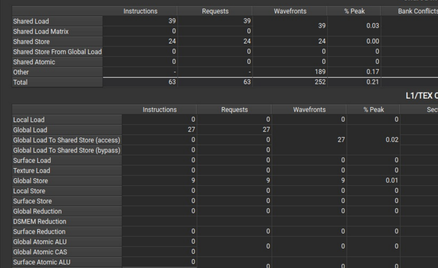
Maximum Sampling Interval [usecond]	1	# Pass Groups	2
Maximum Buffer Size [Mbytes]	4	Dropped Samples [sample]	0

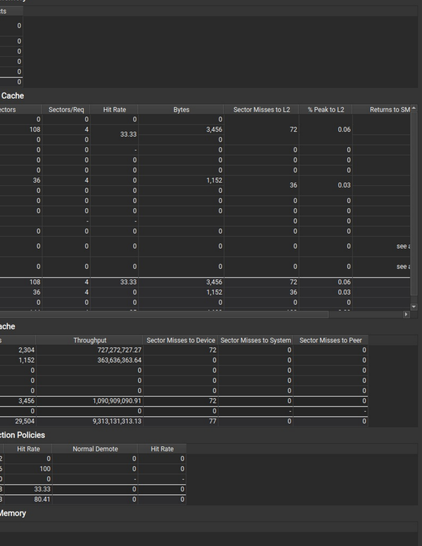
Compute Workload Analysis

Detailed analysis of the compute resources of the streaming multiprocessors (SM), including the achieved instructions per clock (IPC) and the utilization of each available pipeline. Pipelines with very high utilization might limit the overall performance.

Executed Ipc Elapsed [inst/cycle]	0.01	SM Busy [%]	9.37
Executed Ipc Active [inst/cycle]	0.35	Issue Slots Busy [%]	9.37
Issued Ipc Active [inst/cycle]	0.37		

Low Utilization
Est. Local Speedup: 94.03%
All compute pipelines are under-utilized. Either this kernel is very small or it doesn't issue enough warps per scheduler. Check the Launch Statistics and Scheduler Statistics sections for further details.





Memory Workload Analysis

Memory Tables

Detailed analysis of the memory resources of the GPU. Memory can become a limiting factor for the overall kernel performance when fully utilizing the involved hardware units (Mem Busy), exhausting the available communication bandwidth between those units (Max Bandwidth), or by reaching the maximum throughput of issuing memory instructions (Mem Pipes Busy). Detailed chart of the memory units. Detailed tables with data for each memory unit.

Memory Throughput [Gbyte/second]	1.78	Mem Busy [%]	0.59
L1/TEX Hit Rate [%]	25	Max Bandwidth [%]	0.81
L2 Hit Rate [%]	27.22	Mem Pipes Busy [%]	0.46
L2 Compression Success Rate [%]	0	L2 Compression Ratio	0

Shared Memory

	Instructions	Requests	Wavefronts	% Peak	Bank Conflicts
Shared Load	39	39	39	0.03	0
Shared Load Matrix	0	0	0	0	0
Shared Store	24	24	24	0.00	0
Shared Store From Global Load	0	0	0	0	0
Shared Atomic	0	0	0	0	0
Other	-	-	189	0.17	0
Total	63	63	252	0.21	0

L1/TEX Cache

	Instructions	Requests	Wavefronts	% Peak	Sectors	Sectors/Req	Hit Rate	Bytes	Sector Misses to L2	% Peak to L2	Returns to SM
Local Load	0	0	0	0	0	0	0	0			
Global Load	27	27			108	4	33.33	3,456	72	0.06	
Global Load To Shared Store (access)	0	0	27	0.02	0	0		0			
Global Load To Shared Store (bypass)	0	0			0	0	-	0	0	0	
Surface Load	0	0	0	0	0	0	0	0	0	0	
Texture Load	0	0	0	0	0	0	0	0	0	0	
Global Store	9	9	9	0.01	36	4	0	1,152			
Local Store	0	0	0	0	0	0	0	0	36	0.03	
Surface Store	0	0	0	0	0	0	0	0	0	0	
Global Reduction	0	0	0	0	0	0	0	0	0	0	
DSMEM Reduction							-	-	0	0	
Surface Reduction	0	0	0	0	0	0	0	0	0	0	
Global Atomic ALU	0	0	0	0	0	0	0	0	0	0	see
Global Atomic CAS	0										
Surface Atomic ALU	0	0	0	0	0	0	0	0	0	0	see
Surface Atomic CAS	0										
Loads	27	27	27	0.02	108	4	33.33	3,456	72	0.06	
Stores	9	9	9	0.01	36	4	0	1,152	36	0.03	
Atomics & Reductions	0	0	0	0	0	0	0	0	0	0	

L2 Cache

	Requests	Sectors	Sectors/Req	% Peak	Hit Rate	Bytes	Throughput	Sector Misses to Device	Sector Misses to System	Sector Misses to Peer
L1/TEX Load	18	72	4	0.05	0	2,304	727,272,727.27	72	0	0
L1/TEX Store	9	36	4	0.05	100	1,152	363,636,363.64	0	0	0
L1/TEX Atomic ALU	0	0	0	0	0	0	0	0	0	0
L1/TEX Atomic CAS	0	0	0	0	0	0	0	0	0	0
L1/TEX Reduction	0	0	0	0	0	0	0	0	0	0
L1/TEX Total	27	108	4	0.07	33.33	3,456	1,090,909,090.91	72	0	0
ECC Total	0	0	-	0	-	0	0	0	-	-
GPU Total	112	922	8.23	0.59	76.52	29,504	9,313,131,313.13	77	0	0

L2 Cache Eviction Policies

	First	Hit Rate	Last	Hit Rate	Normal	Hit Rate	Normal Demote	Hit Rate
L1/TEX Load	0	0	0	0	72	0	0	0
L1/TEX Store	0	0	0	0	36	100	0	0
L1/TEX Atomic	0	0	0	0	0	0	-	-
L1/TEX Total	0	0	0	0	108	33.33	0	0
GPU Total	171	100	0	0	393	80.41	0	0

Device Memory

	Sectors	% Peak	Bytes	Throughput
Load	176	0.81	5,632	1,777,777,777.78
Store	0	0	0	0
Total	176	0.81	5,632	1,777,777,777.78

Scheduler Statistics

Summary of the activity of the schedulers issuing instructions. Each scheduler maintains a pool of warps that it can issue instructions for. The upper bound of warps in the pool (Theoretical Warps) is limited by the launch configuration. On every cycle each scheduler checks the state of the allocated warps in the pool (Active Warps). Active warps that are not stalled (Eligible Warps) are ready to issue their next instruction. From the set of eligible warps the scheduler selects a single warp from which to issue one or more instructions (Issued Warp). On cycles with no eligible warps, the issue slot is skipped and no instruction is issued. Having many skipped issue slots indicates poor latency hiding.

Active Warps Per Scheduler [warp]	2.25	No Eligible [%]	90.28
Eligible Warps Per Scheduler [warp]	0.12	One or More Eligible [%]	9.72
Issued Warp Per Scheduler	0.10		

Issue Slot Utilization
Est. Local Speedup: 90.28%
Every scheduler is capable of issuing one instruction per cycle, but for this kernel each scheduler only issues an instruction every 10.3 cycles. This might leave hardware resources underutilized and may lead to less optimal performance. Out of the maximum of 12 warps per scheduler, this kernel allocates an average of 2.25 active warps per scheduler, but only an average of 0.12 warps were eligible per cycle. Eligible warps are the subset of active warps that are ready to issue their next instruction. Every cycle with no eligible warp results in no instruction being issued and the issue slot remains unused. To increase the number of eligible warps, reduce the time the active warps are stalled by inspecting the top stall reasons on the Warp State Statistics and Source Counters sections.

Warp State Statistics

Analysis of the states in which all warps spent cycles during the kernel execution. The warp states describe a warp's readiness or inability to issue its next instruction. The warp cycles per instruction define the latency between two consecutive instructions. The higher the value, the more warp parallelism is required to hide this latency. For each warp state, the chart shows the average number of cycles spent in that state per issued instruction. Stalls are not always impacting the overall performance nor are they completely avoidable. Only focus on stall reasons if the schedulers fail to issue every cycle. When executing a kernel with mixed library and user code, these metrics show the combined values.

Warp Cycles Per Issued Instruction [cycle]	23.13	Avg. Active Threads Per Warp	32
Warp Cycles Per Executed Instruction [cycle]	24.60	Avg. Not Predicated Off Threads Per Warp	21.41

Long Scoreboard Stalls
Est. Speedup: 32.51%
On average, each warp of this kernel spends 7.5 cycles being stalled waiting for a scoreboard dependency on a L1TEX (local, global, surface, texture) operation. Find the instruction producing the data being waited upon to identify the culprit. To reduce the number of cycles waiting on L1TEX data accesses verify the memory access patterns are optimal for the target architecture, attempt to increase cache hit rates by increasing data locality (coalescing), or by changing the cache configuration. Consider moving frequently used data to shared memory. This stall type represents about 32.5% of the total average of 23.1 cycles between issuing two instructions.

Warp Stall Check the Warp Stall Sampling (All Samples) table for the top stall locations in your source based on sampling data. The Kernel Profiling Guide provides more details on each stall reason.

Thread Divergence
Est. Speedup: 0.15%
Instructions are executed in warps, which are groups of 32 threads. Optimal instruction throughput is achieved if all 32 threads of a warp execute the same instruction. The chosen launch configuration, early thread completion, and divergent flow control can significantly lower the number of active threads in a warp per cycle. This kernel achieves an average of 32.0 threads being active per cycle. This is further reduced to 21.4 threads per warp due to predication. The compiler may use predication to avoid an actual branch. Instead, all instructions are scheduled, but a per-thread condition code or predicate controls which threads execute the instructions. Try to avoid different execution paths within a warp when possible.

Instruction Statistics

Statistics of the executed low-level assembly instructions (SASS). The instruction mix provides insight into the types and frequency of the executed instructions. A narrow mix of instruction types implies a dependency on few instruction pipelines, while others remain unused. Using multiple pipelines allows hiding latencies and enables parallel execution. Note that 'Instructions/Opcode' and 'Executed Instructions' are measured differently and can diverge if cycles are spent in system calls.

Executed Instructions [inst]	1,116	Avg. Executed Instructions Per Scheduler [inst]	11.62
Issued Instructions [inst]	1,187	Avg. Issued Instructions Per Scheduler [inst]	12.36

FP32 Non-Fused Instructions
Est. Speedup: 0.74%
This kernel executes 18 fused and 90 non-fused FP32 instructions. By converting pairs of non-fused instructions to their fused, higher-throughput equivalent, the achieved FP32 performance could be increased by up to 42% (relative to its current performance). Check the Source page to identify where this kernel executes FP32 instructions.

NVLink Topology

NVLink Topology diagram shows logical NVLink connections with transmit/receive throughput.

NVLink Tables

Detailed tables with properties for each NVLink.

NUMA Affinity

Non-uniform memory access (NUMA) affinities based on compute and memory distances for all GPUs.

Launch Statistics

Summary of the configuration used to launch the kernel. The launch configuration defines the size of the kernel grid, the division of the grid into blocks, and the GPU resources needed to execute the kernel. Choosing an efficient launch configuration maximizes device utilization.

Grid Size	1	Function Cache Configuration	CachePreferNone
Registers Per Thread [register/thread]	16	Static Shared Memory Per Block [byte/block]	0
Block Size	288	Dynamic Shared Memory Per Block [Kbyte/block]	1.15
Threads [thread]	288	Driver Shared Memory Per Block [Kbyte/block]	1.02
Waves Per SM	0.01	Shared Memory Configuration Size [Kbyte]	32.77
Uses Green Context	0	# SMs [SM]	24

Small Grid
Est. Speedup: 95.83%
The grid for this launch is configured to execute only 1 blocks, which is less than the GPU's 24 multiprocessors. This can underutilize some multiprocessors. If you do not intend to execute this kernel concurrently with other workloads, consider reducing the block size to have at least one block per multiprocessor or increase the size of the grid to fully utilize the available hardware resources. See the CUDA Best Practices Guide description for more details on launch configurations.

Occupancy

Occupancy is the ratio of the number of active warps per multiprocessor to the maximum number of possible active warps. Another way to view occupancy is the percentage of the hardware's ability to process warps that is actively in use. Higher occupancy does not always result in higher performance, however, low occupancy always reduces the ability to hide latencies, resulting in overall performance degradation. Large discrepancies between the theoretical and the achieved occupancy during execution typically indicates highly imbalanced workloads.

Theoretical Occupancy [%]	93.75	Block Limit Registers [block]	14
Theoretical Active Warps per SM [warp]	45	Block Limit Shared Mem [block]	15
Achieved Occupancy [%]	18.07	Block Limit Warps [block]	5
Achieved Active Warps Per SM [warp]	8.67	Block Limit SM [block]	24

Achieved Occupancy
Est. Speedup: 80.73%
The difference between calculated theoretical (93.8%) and measured achieved occupancy (18.1%) can be the result of warp scheduling overheads or workload imbalances during the kernel execution. Load imbalances can occur between warps within a block as well as across blocks of the same kernel. See the CUDA Best Practices Guide for more details on optimizing occupancy.

GPU and Memory Workload Distribution

Analysis of workload distribution in active cycles of SM, SMP, SMSP, L1 & L2 caches, and DRAM

Average SM Active Cycles [cycle]	132	Average L1 Active Cycles [cycle]	132
Average L2 Active Cycles [cycle]	283.06	Average SMSP Active Cycles [cycle]	127.21
Average DRAM Active Cycles [cycle]	176	Total SM Elapsed Cycles [cycle]	120,944
Total L1 Elapsed Cycles [cycle]	120,944	Total L2 Elapsed Cycles [cycle]	78,608
Total SMSP Elapsed Cycles [cycle]	483,776	Total DRAM Elapsed Cycles [cycle]	87,040

Source Counters

Source metrics, including branch efficiency and sampled warp stall reasons. Warp Stall Sampling metrics are periodically sampled over the kernel runtime. They indicate when warps were stalled and couldn't be scheduled. See the documentation for a description of all stall reasons. Only focus on stalls if the schedulers fail to issue every cycle.

Branch Instructions [inst]	135	Branch Efficiency [%]	100
Branch Instructions Ratio [%]	0.12	Avg. Divergent Branches	0